🎓 SKILLUP - Prototype Aplikasi Manajemen Kursus Daring

Berikut adalah prototype lengkap aplikasi **SKILLUP** dengan Python, MySQL, dan XAMPP.

📋 Struktur Proyek

```
skillup/
├── main.py              # Entry point aplikasi
├── database.py          # Konfigurasi & helper database
├── auth.py              # Login & register
├── models.py            # Model data (Course, User, Progress)
├── ui_helpers.py        # Helper untuk tampilan CLI/color
├── database_skillup.sql    # Script SQL untuk MySQL
└── requirements.txt
```

🗄️ 1. Database Schema (MySQL)

File: `database_skillup.sql`

```sql
-- Jalankan di phpMyAdmin XAMPP
```

```
CREATE DATABASE IF NOT EXISTS skillup_db;
USE skillup_db;

-- Tabel users
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    role ENUM('siswa', 'instruktur', 'admin') DEFAULT 'siswa',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabel courses
CREATE TABLE courses (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    description TEXT,
    instructor_id INT NOT NULL,
    category VARCHAR(50),
    level ENUM('pemula', 'menengah', 'lanjutan') DEFAULT 'pemula',
    price DECIMAL(10,2) DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (instructor_id) REFERENCES users(id) ON DELETE CASCADE
);
```

```
-- Tabel lessons (materi per kursus)
CREATE TABLE lessons (
    id INT AUTO_INCREMENT PRIMARY KEY,
    course_id INT NOT NULL,
    title VARCHAR(150) NOT NULL,
    content TEXT,
    video_url VARCHAR(255),
    order_num INT DEFAULT 1,
    duration_minutes INT DEFAULT 0,
    FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE
);

-- Tabel enrollments (pendaftaran siswa ke kursus)
CREATE TABLE enrollments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    course_id INT NOT NULL,
    enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status ENUM('aktif', 'selesai', 'batal') DEFAULT 'aktif',
    UNIQUE KEY unique_enroll (user_id, course_id),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE
);

-- Tabel progress (progress per lesson)
```

```
CREATE TABLE progress (
    id INT AUTO_INCREMENT PRIMARY KEY,
    enrollment_id INT NOT NULL,
    lesson_id INT NOT NULL,
    is_completed BOOLEAN DEFAULT FALSE,
    completed_at TIMESTAMP NULL,
    UNIQUE KEY unique_progress (enrollment_id, lesson_id),
    FOREIGN KEY (enrollment_id) REFERENCES enrollments(id) ON DELETE CASCADE,
    FOREIGN KEY (lesson_id) REFERENCES lessons(id) ON DELETE CASCADE
);

-- Seed data: user default
INSERT INTO users (username, password, full_name, email, role) VALUES
('admin', 'admin123', 'Administrator', 'admin@skillup.id', 'admin'),
('instruktur1', 'instruktur123', 'Budi Santoso', 'budi@skillup.id', 'instruktur'),
('siswa1', 'siswa123', 'Ani Wijaya', 'ani@skillup.id', 'siswa');

-- Seed course
INSERT INTO courses (title, description, instructor_id, category, level, price) VALUES
('Python untuk Pemula', 'Belajar Python dari nol sampai bisa bikin program', 2, 'Programming', 'pemula', 0),
('Dasar UI/UX Design', 'Pengenalan design thinking dan tools Figma', 2, 'Design', 'pemula', 150000);

-- Seed lessons
INSERT INTO lessons (course_id, title, content, order_num, duration_minutes) VALUES
(1, 'Pengenalan Python', 'Apa itu Python, instalasi, dan hello world', 1, 30),
```

(1, 'Variabel dan Tipe Data', 'Belajar variabel, integer, string, list', 2, 45),

(1, 'Struktur Kontrol', 'If-else, loop, dan function', 3, 60),

(2, 'Design Thinking', 'Empathize, define, ideate, prototype, test', 1, 40),

(2, 'Pengenalan Figma', 'Tools dasar Figma untuk wireframe', 2, 50);

```

---

## ⚙️ 2. Konfigurasi Database

**File: `database.py`**

```python
import mysql.connector
from mysql.connector import Error

DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "",         # default XAMPP
    "database": "skillup_db"
}

def get_connection():
    try:
        return mysql.connector.connect(**DB_CONFIG)

```
    except Error as e:
        print(f"[DB ERROR] {e}")
        return None

def execute_query(query, params=None, fetch=False, commit=False):
    conn = get_connection()
    if not conn:
        return None
    cursor = conn.cursor(dictionary=True)
    try:
        cursor.execute(query, params or ())
        if commit:
            conn.commit()
        if fetch:
            return cursor.fetchall()
        return cursor.lastrowid
    except Error as e:
        print(f"[QUERY ERROR] {e}")
        return None
    finally:
        cursor.close()
        conn.close()

def init_db_check():
    """Cek koneksi dan tabel-tabel penting"""
    conn = get_connection()
```

```
    if not conn:
        print("❌ Gagal konek ke MySQL. Pastikan XAMPP MySQL berjalan.")
        return False
    cursor = conn.cursor()
    cursor.execute("SHOW TABLES LIKE 'users'")
    if not cursor.fetchone():
        print("⚠️  Tabel belum ada. Import database_skillup.sql dulu di phpMyAdmin.")
        cursor.close(); conn.close()
        return False
    cursor.close(); conn.close()
    return True
```

🎨 3. UI Helper

File: `ui_helpers.py`

```python
import os

def clear():
    os.system('cls' if os.name == 'nt' else 'clear')

def header(title):
```

```
    print("=" * 60)
    print(f"  {title.center(56)}")
    print("=" * 60)

def line():
    print("-" * 60)

def pause():
    input("\nTekan ENTER untuk lanjut...")

def success(msg):
    print(f"✅ {msg}")

def error(msg):
    print(f"❌ {msg}")

def info(msg):
    print(f"ℹ️  {msg}")
```

👤 4. Auth (Login & Register)

File: `auth.py`

```python
from database import execute_query
from ui_helpers import header, line, success, error, pause

def login():
    header("LOGIN SKILLUP")
    username = input("Username : ").strip()
    password = input("Password : ").strip()

    user = execute_query(
        "SELECT * FROM users WHERE username=%s AND password=%s",
        (username, password), fetch=True
    )
    if user:
        success(f"Selamat datang, {user[0]['full_name']} ({user[0]['role']})")
        pause()
        return user[0]
    error("Username / Password salah!")
    pause()
    return None

def register():
    header("REGISTER AKUN BARU")
    username = input("Username    : ").strip()
    password = input("Password    : ").strip()
    full_name = input("Nama Lengkap: ").strip()
```

```
    email = input("Email       : ").strip()
    role = input("Role (siswa/instruktur) [siswa]: ").strip().lower() or "siswa"

    if role not in ("siswa", "instruktur"):
        error("Role tidak valid.")
        pause(); return

    existing = execute_query("SELECT id FROM users WHERE username=%s OR email=%s",
                             (username, email), fetch=True)
    if existing:
        error("Username atau email sudah dipakai.")
        pause(); return

    res = execute_query(
        "INSERT INTO users (username,password,full_name,email,role) VALUES (%s,%s,%s,%s,%s)",
        (username, password, full_name, email, role), commit=True
    )
    if res:
        success("Registrasi berhasil! Silakan login.")
    else:
        error("Registrasi gagal.")
    pause()
```

🧩 5. Models / Business Logic

File: `models.py`

```python
from database import execute_query

# ---------- COURSES ----------
def list_all_courses():
    return execute_query("""
        SELECT c.*, u.full_name AS instructor_name,
               (SELECT COUNT(*) FROM lessons l WHERE l.course_id=c.id) AS total_lessons
        FROM courses c
        JOIN users u ON u.id = c.instructor_id
        ORDER BY c.id DESC
    """, fetch=True) or []

def list_courses_by_instructor(instructor_id):
    return execute_query(
        "SELECT * FROM courses WHERE instructor_id=%s ORDER BY id DESC",
        (instructor_id,), fetch=True) or []

def create_course(title, description, instructor_id, category, level, price):
    return execute_query("""
        INSERT INTO courses (title,description,instructor_id,category,level,price)
        VALUES (%s,%s,%s,%s,%s,%s)
```

```
    """, (title, description, instructor_id, category, level, price), commit=True)

def get_course(course_id):
    res = execute_query("SELECT * FROM courses WHERE id=%s", (course_id,), fetch=True)
    return res[0] if res else None

def delete_course(course_id):
    return execute_query("DELETE FROM courses WHERE id=%s", (course_id,), commit=True)

# ---------- LESSONS ----------
def list_lessons(course_id):
    return execute_query(
        "SELECT * FROM lessons WHERE course_id=%s ORDER BY order_num",
        (course_id,), fetch=True) or []

def add_lesson(course_id, title, content, video_url, order_num, duration):
    return execute_query("""
        INSERT INTO lessons (course_id,title,content,video_url,order_num,duration_minutes)
        VALUES (%s,%s,%s,%s,%s,%s)
    """, (course_id, title, content, video_url, order_num, duration), commit=True)

# ---------- ENROLLMENT ----------
def enroll(user_id, course_id):
    existing = execute_query(
        "SELECT id FROM enrollments WHERE user_id=%s AND course_id=%s",
        (user_id, course_id), fetch=True)
```

```
    if existing:
        return False, "Sudah terdaftar di kursus ini."
    execute_query(
        "INSERT INTO enrollments (user_id,course_id) VALUES (%s,%s)",
        (user_id, course_id), commit=True)
    return True, "Berhasil mendaftar kursus."

def my_courses(user_id):
    return execute_query("""
        SELECT e.id AS enrollment_id, c.*,
            (SELECT COUNT(*) FROM lessons WHERE course_id=c.id) AS total_lessons,
            (SELECT COUNT(*) FROM progress p
             WHERE p.enrollment_id=e.id AND p.is_completed=1) AS completed_lessons
        FROM enrollments e
        JOIN courses c ON c.id = e.course_id
        WHERE e.user_id=%s AND e.status='aktif'
    """, (user_id,), fetch=True) or []

def get_enrollment(enrollment_id):
    res = execute_query("SELECT * FROM enrollments WHERE id=%s", (enrollment_id,),
fetch=True)
    return res[0] if res else None

# ---------- PROGRESS ----------
def get_progress_map(enrollment_id):
    rows = execute_query("""
```

```
    SELECT lesson_id, is_completed FROM progress WHERE enrollment_id=%s
    """, (enrollment_id,), fetch=True) or []
    return {r['lesson_id']: bool(r['is_completed']) for r in rows}

def mark_lesson(enrollment_id, lesson_id, completed=True):
    # upsert
    existing = execute_query(
        "SELECT id FROM progress WHERE enrollment_id=%s AND lesson_id=%s",
        (enrollment_id, lesson_id), fetch=True)
    if existing:
        execute_query("""
            UPDATE progress SET is_completed=%s,
                completed_at = IF(%s, NOW(), NULL)
            WHERE enrollment_id=%s AND lesson_id=%s
        """, (completed, completed, enrollment_id, lesson_id), commit=True)
    else:
        execute_query("""
            INSERT INTO progress (enrollment_id,lesson_id,is_completed,completed_at)
            VALUES (%s,%s,%s, IF(%s, NOW(), NULL))
        """, (enrollment_id, lesson_id, completed, completed), commit=True)

    # update status enrollment jika semua lesson selesai
    total = execute_query("""
        SELECT COUNT(*) AS n FROM lessons WHERE course_id =
        (SELECT course_id FROM enrollments WHERE id=%s)
    """, (enrollment_id,), fetch=True)[0]['n']
```

```
    done = execute_query("""
        SELECT COUNT(*) AS n FROM progress
        WHERE enrollment_id=%s AND is_completed=1
    """, (enrollment_id,), fetch=True)[0]['n']

    status = 'selesai' if total > 0 and done == total else 'aktif'
    execute_query("UPDATE enrollments SET status=%s WHERE id=%s",
                  (status, enrollment_id), commit=True)
    return done, total
```

🖥️ 6. Menu Siswa

Tambahkan dalam **`main.py`** (lihat bagian 8), fungsi-fungsi berikut bisa ditaruh di `menus.py` atau langsung di `main.py`.

```python
from models import *
from ui_helpers import header, line, success, error, info, pause, clear

def menu_siswa(user):
    while True:
        clear()
        header(f"DASHBOARD SISWA - {user['full_name']}")
```

```
        print("1. Lihat Semua Kursus")
        print("2. Daftar Kursus")
        print("3. Kursus Saya & Progress")
        print("4. Logout")
        line()
        pilihan = input("Pilih menu: ").strip()

        if pilihan == "1":
            lihat_semua_kursus()
        elif pilihan == "2":
            daftar_kursus(user)
        elif pilihan == "3":
            kursus_saya(user)
        elif pilihan == "4":
            return
        else:
            error("Menu tidak valid."); pause()

def lihat_semua_kursus():
    clear(); header("SEMUA KURSUS")
    courses = list_all_courses()
    if not courses:
        info("Belum ada kursus."); pause(); return
    for c in courses:
        print(f"[{c['id']}] {c['title']}")
        print(f"    Kategori : {c['category']} | Level: {c['level']}")
```

```
        print(f"    Instruktur: {c['instructor_name']}")
        print(f"    Materi   : {c['total_lessons']} lesson")
        print(f"    Harga    : Rp {c['price']:,.0f}")
        line()
    pause()

def daftar_kursus(user):
    clear(); header("DAFTAR KURSUS")
    lihat_semua_kursus_compact()
    try:
        cid = int(input("ID kursus yang ingin diikuti: "))
    except ValueError:
        error("ID tidak valid."); pause(); return
    ok, msg = enroll(user['id'], cid)
    (success if ok else error)(msg)
    pause()

def lihat_semua_kursus_compact():
    for c in list_all_courses():
        print(f"[{c['id']}] {c['title']} ({c['level']}) - {c['total_lessons']} lesson")
    line()

def kursus_saya(user):
    while True:
        clear(); header("KURSUS SAYA")
        courses = my_courses(user['id'])
```

```
        if not courses:
            info("Anda belum mendaftar kursus apa pun."); pause(); return
        for c in courses:
            total = c['total_lessons'] or 1
            done = c['completed_lessons']
            pct = int(done / total * 100)
            bar = "" * (pct // 10) + "" * (10 - pct // 10)
            print(f"[{c['enrollment_id']}] {c['title']}")
            print(f"    Progress: [{bar}] {pct}% ({done}/{c['total_lessons']})")
            line()
        choice = input("\nMasukkan enrollment ID untuk buka materi (0 = kembali): ").strip()
        if choice == "0" or not choice:
            return
        try:
            eid = int(choice)
        except ValueError:
            continue
        buka_kursus(user, eid)

def buka_kursus(user, enrollment_id):
    enroll_data = get_enrollment(enrollment_id)
    if not enroll_data or enroll_data['user_id'] != user['id']:
        error("Enrollment tidak ditemukan."); pause(); return

    course = get_course(enroll_data['course_id'])
    lessons = list_lessons(course['id'])
```

```
    progress = get_progress_map(enrollment_id)

    while True:
        clear(); header(f"MATERI: {course['title']}")
        for l in lessons:
            status = " ✅ " if progress.get(l['id']) else " ⬜ "
            print(f"{status} [{l['id']}] (Pertemuan {l['order_num']}) {l['title']} "
                  f"- {l['duration_minutes']} menit")
        line()
        print("Ketik ID lesson untuk buka, atau 0 untuk kembali.")
        choice = input(">> ").strip()
        if choice == "0":
            return
        try:
            lid = int(choice)
        except ValueError:
            continue
        lesson = next((x for x in lessons if x['id'] == lid), None)
        if not lesson:
            error("Lesson tidak ditemukan."); pause(); continue
        buka_lesson(enrollment_id, lesson, progress)

def buka_lesson(enrollment_id, lesson, progress):
    clear(); header(lesson['title'])
    print(f"Durasi: {lesson['duration_minutes']} menit\n")
    print(lesson['content'] or "(tidak ada konten)")
```

```
    if lesson['video_url']:
        print(f"\n 🎬 Video: {lesson['video_url']}")
    line()
    current = progress.get(lesson['id'], False)
    print(f"Status: {'✅ Selesai' if current else '⬜ Belum selesai'}")
    print("1. Tandai Selesai")
    print("2. Tandai Belum Selesai")
    print("3. Kembali")
    p = input("Pilih: ").strip()
    if p == "1":
        done, total = mark_lesson(enrollment_id, lesson['id'], True)
        success(f"Progress diperbarui ({done}/{total}).")
        progress[lesson['id']] = True
        pause()
    elif p == "2":
        mark_lesson(enrollment_id, lesson['id'], False)
        success("Ditandai belum selesai.")
        progress[lesson['id']] = False
        pause()
```

👩‍🏫 7. Menu Instruktur

```python
```

```
def menu_instruktur(user):
    while True:
        clear()
        header(f"DASHBOARD INSTRUKTUR - {user['full_name']}")
        print("1. Kursus Saya")
        print("2. Buat Kursus Baru")
        print("3. Kelola Materi Kursus")
        print("4. Lihat Statistik Siswa")
        print("5. Logout")
        line()
        p = input("Pilih menu: ").strip()

        if p == "1": lihat_kursus_instruktur(user)
        elif p == "2": buat_kursus(user)
        elif p == "3": kelola_materi(user)
        elif p == "4": statistik_siswa(user)
        elif p == "5": return
        else: error("Menu tidak valid."); pause()

def lihat_kursus_instruktur(user):
    clear(); header("KURSUS SAYA")
    courses = list_courses_by_instructor(user['id'])
    if not courses:
        info("Belum ada kursus."); pause(); return
    for c in courses:
        print(f"[{c['id']}] {c['title']} ({c['level']}) - Rp {c['price']:,.0f}")
```

```
    pause()


def buat_kursus(user):
    clear(); header("BUAT KURSUS BARU")
    title = input("Judul       : ").strip()
    desc  = input("Deskripsi   : ").strip()
    cat   = input("Kategori    : ").strip()
    lvl   = input("Level (pemula/menengah/lanjutan) [pemula]: ").strip().lower() or "pemula"
    try:
        price = float(input("Harga (0=Gratis): ").strip() or 0)
    except ValueError:
        price = 0
    res = create_course(title, desc, user['id'], cat, lvl, price)
    if res:
        success(f"Kursus dibuat (ID={res}).")
    else:
        error("Gagal membuat kursus.")
    pause()


def kelola_materi(user):
    clear(); header("KELOLA MATERI")
    courses = list_courses_by_instructor(user['id'])
    if not courses:
        info("Belum ada kursus."); pause(); return
    for c in courses:
        print(f"[{c['id']}] {c['title']}")
```

```
try:
    cid = int(input("\nPilih ID kursus: "))
except ValueError:
    error("ID tidak valid."); pause(); return
if not any(c['id'] == cid for c in courses):
    error("Bukan kursus Anda."); pause(); return


while True:
    clear(); header(f"MATERI KURSUS #{cid}")
    lessons = list_lessons(cid)
    for l in lessons:
        print(f"[{l['id']}] Pertemuan {l['order_num']}: {l['title']} ({l['duration_minutes']} mnt)")
    if not lessons: info("Belum ada materi.")
    line()
    print("1. Tambah Materi")
    print("2. Kembali")
    p = input("Pilih: ").strip()
    if p == "2": return
    if p == "1":
        title = input("Judul materi: ").strip()
        content = input("Konten      : ").strip()
        video = input("URL video   : ").strip()
        try:
            order = int(input(f"Urutan (default {len(lessons)+1}): ").strip() or (len(lessons)+1))
            dur = int(input("Durasi menit: ").strip() or 0)
        except ValueError:
```

```
            error("Input angka salah."); pause(); continue
        add_lesson(cid, title, content, video, order, dur)
        success("Materi ditambahkan."); pause()


def statistik_siswa(user):
    clear(); header("STATISTIK SISWA PER KURSUS")
    courses = list_courses_by_instructor(user['id'])
    if not courses:
        info("Belum ada kursus."); pause(); return
    for c in courses:
        rows = execute_query("""
            SELECT u.full_name, e.status,
             (SELECT COUNT(*) FROM progress p WHERE p.enrollment_id=e.id AND
p.is_completed=1) AS done,
             (SELECT COUNT(*) FROM lessons l WHERE l.course_id=e.course_id) AS total
            FROM enrollments e JOIN users u ON u.id=e.user_id
            WHERE e.course_id=%s
        """, (c['id'],), fetch=True) or []
        print(f"\n📘 {c['title']} ({len(rows)} siswa)")
        for r in rows:
            total = r['total'] or 1
            pct = int(r['done'] / total * 100)
            print(f"  - {r['full_name']}: {r['done']}/{r['total']} ({pct}%) [{r['status']}]")
    pause()
```

🚀 8. Main Program

File: `main.py`

```python
from database import init_db_check
from ui_helpers import clear, header, line, error, info, pause
from auth import login, register

# import menu functions
from menus import menu_siswa, menu_instruktur

APP_NAME = "SKILLUP"
APP_TAGLINE = "Manajemen Kursus Daring & Progress"

def welcome():
    clear()
    header(f"{APP_NAME} - {APP_TAGLINE}")
    print("1. Login")
    print("2. Register")
    print("3. Keluar")
    line()

def main():
```

```
    if not init_db_check():
        pause()
        return

    while True:
        welcome()
        pilih = input("Pilih menu: ").strip()
        if pilih == "1":
            user = login()
            if user:
                if user['role'] == 'siswa':
                    menu_siswa(user)
                elif user['role'] == 'instruktur':
                    menu_instruktur(user)
                elif user['role'] == 'admin':
                    print("👑 Admin login - fitur admin dapat ditambahkan.")
                    pause()
        elif pilih == "2":
            register()
        elif pilih == "3":
            info("Terima kasih telah menggunakan SKILLUP 👋")
            break
        else:
            error("Menu tidak valid."); pause()

if __name__ == "__main__":
```

```
main()
```

📦 9. Requirements

File: `requirements.txt`

```
mysql-connector-python
```

Install:

```bash
pip install mysql-connector-python
```

▶️ Cara Menjalankan

1. **Start XAMPP** → jalankan **Apache** (opsional) & **MySQL**
2. Buka **phpMyAdmin** (`http://localhost/phpmyadmin`)
3. Import `database_skillup.sql`
4. Install library:
   ```bash
   ```

pip install mysql-connector-python

```

5. Jalankan aplikasi:

```bash

python main.py

```

Akun Default (seed)

| Username | Password | Role |

|---|---|---|

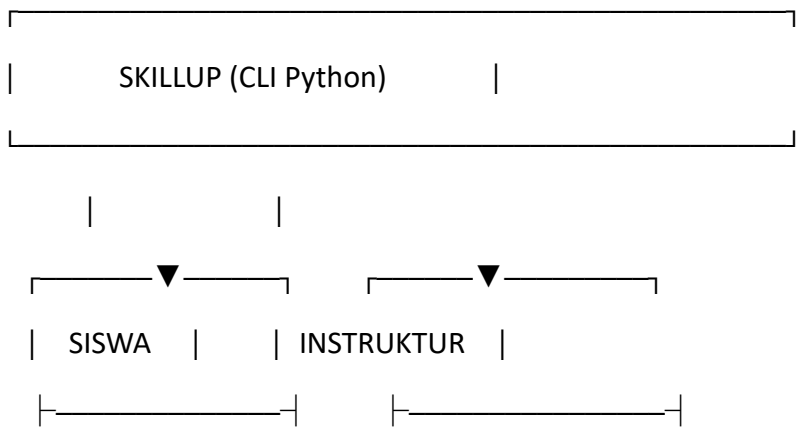
| admin | admin123 | admin |

| instruktur1 | instruktur123 | instruktur |

| siswa1 | siswa123 | siswa |

🧭 Alur Fitur Utama

```



```
• Lihat katalog kursus		• Buat kursus
• Daftar kursus		• Tambah lesson
• Lihat progress		• Statistik
• Tandai lesson		siswa
selesai (auto %	└────────────────┘
progress & status
'selesai')
  └──────────────────────────┘
```

🎯 Fitur yang Sudah Berjalan

Fitur	Deskripsi
🔐 **Autentikasi**	Login & Register dengan role (siswa / instruktur / admin)
📚 **Katalog Kursus**	Semua kursus dengan info instruktur, level, harga, jumlah lesson
✍️ **Enrollment**	Siswa mendaftar kursus (validasi duplikat)
📈 **Progress Tracking**	Per-lesson ✓, progress bar %, auto-update status enrollment ke `selesai`
👩‍🏫 **Instructor Panel**	Buat kursus, tambah materi, lihat statistik siswa
🗄️ **Database Relasional**	5 tabel dengan foreign key & constraint

---

💡 Pengembangan Lanjutan (opsional)

- **GUI**: ganti CLI → **Tkinter / PyQt** / **Flask + HTML** (web)
- **Export sertifikat PDF** dengan `reportlab` saat kursus selesai
- **Quiz & scoring** (tabel baru `quizzes`, `quiz_results`)
- **Rating & review** kursus
- **Notifikasi email** (SMTP) saat progress 100%
- **Dashboard admin** (approval instruktur, manajemen user)

Silakan jalankan dan sesuaikan. Jika butuh versi **GUI (Tkinter)** atau **Web (Flask)**, tinggal bilang! 🚀